

Очистка данных

Цель видео

Научиться очищать данные с помощью ETL-процедур.

Очистка данных

Очистка данных — важный этап подготовки данных для анализа и машинного обучения. Методы очистки данных:

- удаление дубликатов
- обработка пропущенных значений
- обработка выбросов
- корректировка ошибок в данных

Пример

Удаление дубликатов:

```
df.drop_duplicates(inplace=True)
```

Заполнение пропущенных значений:

```
df['age'].fillna(df['age'].mean(),  
inplace=True)  
df['name'].fillna('Unknown', inplace=True)
```

Обработка выбросов:

```
Q1 = df['income'].quantile(0.25)  
Q3 = df['income'].quantile(0.75)  
IQR = Q3 - Q1  
lower_bound = Q1 - 1.5 * IQR  
upper_bound = Q3 + 1.5 * IQR  
df = df[(df['income'] >= lower_bound) &  
(df['income'] <= upper_bound)]
```

Полный код

```
import pandas as pd

# Пример данных
data = {'name': ['Alice', 'Bob', 'Charlie', 'David', 'Edward', 'Frank', 'Grace'],
        'age': [25, 30, 35, None, 45, 50, 100],
        'income': [50000, 54000, 58000, 62000, 66000, 70000, 1000000]}
df = pd.DataFrame(data)

# Удаление дубликатов
df.drop_duplicates(inplace=True)

# Заполнение пропущенных значений
df['age'].fillna(df['age'].mean(), inplace=True)
df['name'].fillna('Unknown', inplace=True)

# Обработка выбросов
Q1 = df['income'].quantile(0.25)
Q3 = df['income'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df = df[(df['income'] >= lower_bound) & (df['income'] <= upper_bound)]

print(df)
```


Output

name	age	income
Alice	25	50000
Bob	30	54000
Charlie	35	NaN
None	NaN	62000

Данные **ДО** очистки

VS

	name	age	income
0	Alice	25	50000
1	Bob	30	54000
2	Charlie	35	58000
3	Edward	45	66000
4	Frank	50	70000

После

Примеры ETL-процедур

```
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from datetime import datetime, timedelta
import pandas as pd
```

```
default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'start_date': datetime(2023, 7, 1),
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}
```

```
dag = DAG(
    'data_cleaning_pipeline',
    default_args=default_args,
    description='ETL-процедура для очистки данных',
    schedule_interval=timedelta(days=1),)
```

```
.....
```

Использование Apache Airflow для автоматизации ETL

Этот код выполняет функцию извлечения данных из файла CSV в рамках процесса ETL.

```
def extract(**kwargs):  
    df = pd.read_csv('/path/to/data.csv')  
    return df
```

Функция **extract(**kwargs)**: принимает неограниченное количество аргументов в виде словаря (kwargs).

pd.read_csv('/path/to/data.csv'): read_csv библиотека Pandas для чтения данных из файла CSV.

return df: возвращает созданный DataFrame df, содержащий данные из файла CSV.

Использование Apache Airflow для автоматизации ETL

Этот код выполняет этап трансформации данных в процессе ETL.

```
def transform(**kwargs):  
    ti = kwargs['ti']  
    df = ti.xcom_pull(task_ids='extract')  
    df.drop_duplicates(inplace=True)  
    df['age'].fillna(df['age'].mean(), inplace=True)  
    df['name'].fillna('Unknown', inplace=True)  
    Q1 = df['income'].quantile(0.25)  
    Q3 = df['income'].quantile(0.75)  
    IQR = Q3 - Q1  
    lower_bound = Q1 - 1.5 * IQR  
    upper_bound = Q3 + 1.5 * IQR  
    df = df[(df['income'] >= lower_bound) & (df['income'] <=  
upper_bound)]  
    return df
```

Извлечение Task Instance (ti) из kwargs.

Извлечение данных с помощью XCom.

Удаление дубликатов.

Заполнение пропущенных значений в столбце **age**.

Заполнение пропущенных значений в столбце **name**.

Фильтрация выбросов.

Использование Apache Airflow для автоматизации ETL

Этот код выполняет этап загрузки данных в процессе ETL.

```
def load(**kwargs):  
    ti = kwargs['ti']  
    df = ti.xcom_pull(task_ids='transform')  
    df.to_csv('/path/to/cleaned_data.csv',  
index=False)
```

Извлечение Task Instance (ti) из kwargs.

Извлечение данных с помощью XCom.

Удаление дубликатов.

Сохранение данных в CSV-файл.

Параметр **index=False** указывает, что индексы **DataFrame** не должны быть записаны в **CSV**-файл.

Пример DAG (Directed Acyclic Graph) в Airflow

Объяснение DAG:

extract_task: извлекает данные из CSV-файла.

transform_task: преобразует данные, очищая их от дубликатов, заполняя пропущенные значения и обрабатывая выбросы.

load_task: загружает очищенные данные в новый CSV-файл.

[Полный Python-скрипт для автоматизации ETL-процедур](#)